



Title

scest — Estimation for Synthetic Control Methods.

Syntax

```
scest , dfname(string) [p(#) direc(string) Q(#) lb(#) name(string) V(string)
opt(string) pypinoccheck]
```

```
{p_end}
```

Description

scest implements estimation procedures for Synthetic Control (SC) methods using least squares, lasso, ridge, or simplex-type constraints according to [Cattaneo, Feng, and Titiunik \(2021\)](#) and for a single treated unit and [Cattaneo, Feng, Palomba, and Titiunik \(2027\)](#) for multiple treated units and staggered adoption. The command is a wrapper of the companion Python package. As such, the user needs to have a running version of Python with the package installed. A tutorial on how to install Python and link it to Stata can be found [here](#).

Companion R and Python packages are described in [Cattaneo, Feng, Palomba and Titiunik \(2025\)](#).

Companion commands are: [scdata](#) for data preparation, [scpi](#) for inference procedures, and [scplot](#) for SC plots.

Related Stata, R, and Python packages useful for inference in SC designs are described in the following website:

<https://nppackages.github.io/scpi/>

For an introduction to synthetic control methods, see [Abadie \(2021\)](#) and references therein.

Options

dfname(string) specifies the name of the Python object containing the processed data created with [scdata](#).

Loss Function and Constraints

These options let the user specify the type of constraint to be imposed to estimate the SC weights and the loss function. The user controls the lower bound on the weights (option **lb**), the norm of the weights to be constrained (option **p**), the direction of the constraint on the norm (option **dir**), the size of the constraint on the norm (option **q**), and the shape of the weighting matrix in the loss function (option **V**). Alternatively, some popular constraints can be selected through the option **name**. A detailed description of the popular constraints implemented can be found in [Cattaneo, Feng, Palomba and Titiunik \(2025\)](#).

lb(#) specifies the lower bound on the weights. The default is **lb(0)**.

p(#) sets the type of norm to be constrained. Options are:

- 0 no constraint on the norm of the weights is imposed.
- 1 a constraint is imposed on the L1 norm of the weights (the default).
- 2 a constraint is imposed on the L2 norm of the weights.

direc(string) specifies the direction of the constraint on the norm of the weights. Options are:

- <= the constraint on the norm of the weights is an inequality constraint.
- = the constraint on the norm of the weights is an equality constraint (the default).

Q(#) specifies the size of the constraint on the norm of the weights.

name(string) specifies the name of the constraint to be used. Options are:

- simplex** classic synthetic control estimator where the weights are constrained to be non-negative and their L1 norm must be equal to 1.
- lasso** weights are estimated using a Lasso-type penalization
- ridge** weights are estimated using a Ridge-type penalization.
- L1-L2** weights are estimated using a Simplex-type constraint and a Ridge-type penalization.
- ols** weights are estimated without constraints using least squares

V(string) specifies the weighting matrix to be used in the loss function. The default is the identity matrix (option **V("separate")**), so equal weight is given to all observations. The other possibility is to specify **V("pooled")** for the pooled fit.

Others

opt(string) a string specifying the stopping criteria used by the underlying optimizer (**nlopt**) for point estimation. The default is a sequential quadratic programming (SQP) algorithm for nonlinearly constrained gradient-based optimization ('SLSQP'). The default value is **opt('maxeval' = 5000, 'xtol_rel' = 1e-8, 'xtol_abs' = 1e-8, 'ftol_rel' = 1e-12, 'ftol_abs' = 1e-12, 'tol_eq' = 1e-8, 'tol_ineq' = 1e-8)**. In case a lasso-type constraint is implemented, a different optimizer (**cvxpy**) is used and stopping criteria cannot be changed.

pypinocheck if specified avoids to check that the version of **scpi_pkg** in Python is the one required by **scest** in Stata. When not specified performs the check and stores a macro called to avoid checking it multiple times.

Example: Germany Data

```
Setup
    . use scpi_germany.dta

Prepare data
    . scdata gdp, dfname("python_scddata") id(country) outcome(gdp) time(year)
      treatment(status) cointegrated

Estimate Synthetic Control with a simplex constraint
    . scest, dfname("python_scddata") name(simplex)
```

Stored results

scest stores the following in **e()**:

Scalars

e(KMI)	number of covariates used for adjustment
e(I)	number of treated units

Macros

e(outcomevar)	name of outcome variable
e(features)	name of features
e(constant)	logical indicating the presence of a common constant across features
e(anticipation)	logical indicating the extent of anticipation effects
e(donors)	donor units for each treated unit
e(cointegrated_data)	logical indicating cointegration
e(p)	type of norm of the weights used in constrained estimation
e(dir)	direction of the constraint on the norm of the weights
e(name)	name of constraint used in estimation

Matrices

e(A)	pre-treatment features of the treated unit
e(B)	pre-treatment features of the control units
e(C)	covariates used for adjustment
e(P)	covariates used to predict the out-of-sample series for the synthetic unit
e(pred)	predicted values of the features of the treated unit
e(res)	residuals e(A) - e(pred)
e(w)	weights of the controls
e(r)	coefficients of the covariates used for adjustment
e(beta)	stacked version of e(w) and e(r)
e(Y_pre)	pre-treatment outcome of the treated unit (only returned if one treated unit)
e(Y_post)	post-treatment outcome of the treated unit (only returned if one treated unit)
e(Y_pre_fit)	estimate pre-treatment outcome of the treated unit
e(Y_post_fit)	estimated post-treatment outcome of the treated unit
e(T0)	number of pre-treatment periods per feature for each treated unit
e(M)	number of features for each treated unit
e(KM)	number of covariates used for adjustment for each treated unit
e(J)	number of donors for each treated unit
e(T1)	number of post-treatment periods for each treated unit
e(Qstar)	regularized constraint on the norm

References

- Abadie, A. 2021. Using synthetic controls: Feasibility, data requirements, and methodological aspects. *Journal of Economic Literature*, 59(2), 391-425.
- Cattaneo, M. D., Feng, Y., and Titiunik, R. 2021. Prediction intervals for synthetic control methods. *Journal of the American Statistical Association*, 116(536), 1865-1880.
- Cattaneo, M. D., Feng, Y., Palomba, F., and Titiunik, R. 2025. scpi: Uncertainty Quantification for Synthetic Control Methods. *Journal of Statistical Software*, 113(1), 1-38.
- Cattaneo, M. D., Feng, Y., Palomba, F., and Titiunik, R. 2027. Uncertainty Quantification in Synthetic Controls with Staggered Treatment Adoption. *Review of Economics and Statistics*, forthcoming.

Authors

Matias D. Cattaneo, Princeton University. matias.d.cattaneo@gmail.com.

Yingjie Feng, Tsinghua University. fengyingjiepk@gmail.com.

Filippo Palomba, Princeton University. filippo.palomba19@gmail.com.

Rocio Titiunik, Princeton University. rocio.titiunik@gmail.com.